

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

PROCEDURAL GENERATION OF TREE MODELS
BAKALÁRSKA PRÁCA

2025
SAMUEL BLEŽÁK

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

PROCEDURAL GENERATION OF TREE MODELS
BAKALÁRSKA PRÁCA

Študijný program: Aplikovaná informatika
Študijný odbor: Aplikovaná informatika
Školiace pracovisko: Katedra aplikovanej informatiky
Školiteľ: doc. RNDr. Roman Durikovič, PhD.

Bratislava, 2025
Samuel Bležák



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta:

Študijný program:

Študijný odbor:

Typ záverečnej práce:

Jazyk záverečnej práce:

Sekundárny jazyk:

Názov:

Anotácia:

Vedúci:

Katedra:

Vedúci katedry:

Dátum zadania:

Dátum schválenia:

garant študijného programu

.....
študent

.....
vedúci práce

PodĎakovanie: Tu môžete poďakovať školiteľovi, prípadne ďalším osobám, ktoré vám s prácou nejako pomohli, poradili, poskytli dáta a podobne.

Abstrakt

Táto práca sa zameriava na pokročilé techniky modelovania stromov v počítačovej grafike, pričom skúma nový spôsob, ako vytvárať realistické a rôznorodé virtuálne stromy. Hlavným cieľom bolo analyzovať a otestovať metódu, ktorá spája biologicky inšpirované princípy s možnosťami interaktívneho návrhu. Na dosiahnutie týchto cieľov boli použité techniky založené na definovaní objemu konárov pomocou prameňov a simulácii prirodzeného rastu vetiev. Výsledky ukazujú, že tento prístup dokáže vytvoriť modely stromov, ktoré sú nielen vizuálne presvedčivé, ale umožňujú aj zobrazenie zložitých tvarov, ako sú zvraty či diery v kmeni a konároch. Tieto zistenia prinášajú prínos do oblasti informatiky, pretože ponúkajú nový nástroj na tvorbu realistických prírodných scén, ktorý môže nájsť uplatnenie v počítačových hrách, filmoch či plánovaní miest a parkov. Práca tak rozširuje možnosti dizajnu virtuálneho prostredia a ukazuje, ako môžu jednoduché princípy viesť k prekvapivo zložitým a užitočným výsledkom.

Kľúčové slová: počítačová grafika, model stromov, generovanie

Abstract

This work focuses on advanced tree modeling techniques in computer graphics, exploring a new way to create realistic and virtual trees. The main goal was to analyze and test a method that combines biologically inspired principles with the possibility of interactive design. Techniques based on defining the volume of trees using springs and simulating natural branch growth were used to achieve these goals. The results show that this approach can create tree models that are not only visually convincing, but also allow the discovery of complex shapes, such as twists and holes in trunks and branches. These findings bring benefits to the field of computer science, as they offer a new tool for creating realistic natural scenes that can find application in computer games, films, or city and park planning. The work thus expands the possibilities of virtual environment design and shows that the principles can lead to surprisingly complex simple and useful results.

Keywords: computer graphics, tree model, generation

Obsah

1	Úvod	1
2	Súvisiace práce	3
2.1	Metódy založené na pravidlách a gramatikách	3
2.2	Procedurálne modelovanie	4
2.3	Biologicky motivované modely	4
2.4	Strand-based a volumetrické modely	5
2.5	Rekonštrukcia stromov zo vstupných dát	6
2.6	Zhrnutie a pozícia našej práce	6
3	Teoretický základ	7
3.1	L-systémy	7
3.1.1	Formálna definícia	7
3.1.2	Príklad derivácie	8
3.1.3	Stochastické L-systémy	9
3.2	Korytnačia grafika v 3D	10
3.2.1	Stav korytnačky	10
3.2.2	Súbor príkazov	10
3.3	Potrubný model	11
3.4	Vyhladzovanie polomerov	12
4	Návrh systému	15
5	Implementácia	17
6	Výsledky práce	19
7	LaTeX	21
7.1	Obrázky	21
	Záver	23
	Príloha A	27

Zoznam obrázkov

3.1	Vývoj L-systému pre pravidlo $F \rightarrow F[+F]F[-F]F$ s uhlom rotácie $\delta = 45$ v 2D priestore.	9
7.1	Ukážka hry Červík	22

Zoznam tabuliek

3.1	Interpretácia symbolov L-systému korytnačkou v 3D priestore.	11
7.1	Doba výpočtu a operačná pamäť potrebná na spracovanie vstupu XYZ	22

Kapitola 1

Úvod

Stromy a rastliny patria medzi najdôležitejšie a najčastejšie využívané vizuálne prvky v počítačovej grafike. Ich využitie si môžeme všímať v rôznych odvetviach ako napríklad vo videohrách, filmoch, aplikáciách virtuálnej a rozšírenej reality, simulátoroch alebo architektonických vizualizáciách. S narastajúcimi nárokmi na rozsah a detail virtuálnych prostredí sa ručné modelovanie veľkého množstva rôznorodých realistických stromov stáva časovo a pracovne náročné.

Tieto dôvody viedli k využitiu procedurálnych metód generovania modelov stromov, ktoré umožňujú efektívnejšie vytvárať ich viacero variácií podľa požiadaviek užívateľa, a to z hľadiska času aj nákladov. Na rozdiel od ručne modelovaných stromov sú procedurálne modely založené na logických a matematických pravidlách, čo výrazne uľahčuje simuláciu ich rastu a správania v prírode. Umožňujú napríklad modelovať adaptáciu na okolité prostredie, ako je reakcia na svetelné podmienky (fototropizmus) alebo fyzikálne kolízie s inými objektmi.

Cieľom tejto bakalárskej práce je implementovať program, ktorý by umožnil užívateľovi procedurálne generovať modely stromov a následne upravovať ich vlastnosti (napr. hrúbka kmeňa, dĺžka konárov, hustota listov). Užívateľ bude taktiež môcť interaktívne ovplyvňovať vývoj stromu tromi spôsobmi - injekciami živín pre lokálny rast, orezávaním vetiev a tvorbou pukov. Program taktiež bude ponúkať 3D zobrazenie vygenerovaných stromov a umožní užívateľovi exportovať strom v .OBJ súboroch.

Implementácia je realizovaná v aplikácii Unity v jazyku C# s použitím vlastného editora. Systém implementácie je založený na kombinácii stochastických L-systémov na generovanie kostry stromu, 3D korytnačej grafiky na interpretáciu kostry stromu v 3D priestore a modelu rastu založeného na vitalite (vigor-based growth model) inšpirovaného Borchert-Hondovou teóriou, ktorý riadi distribúciu rastových zdrojov. Geometria stromu je vykreslená ako polygonálna sieť (mesh) vytvorená z valcovitých segmentov s plynulými prechodmi hrúbky, doplnený o procedurálne generované textúry kôry a listov.

Práce je rozdělena do

Kapitola 2

Súvisiace práce

Modelovanie stromov a rastlín v počítačovej grafike má korene už v práci Lindenmayera z konca 60. rokov 20. storočia, no výraznejší rozvoj tejto oblasti nastal až v priebehu 70. a 80. rokov, keď vznikli prvé gramatické a parametrické modely vetviacich štruktúr. Odvtedy vzniklo množstvo prístupov, ktoré k problému pristupujú z rôznych uhlov — od čisto geometrických metód a metód založených na pravidlách cez biologicky motivované simulácie až po moderné metódy založené na dátach. Táto kapitola poskytuje prehľad najvýznamnejších metód a zároveň ukazuje, kam v kontexte existujúcich prístupov zapadá náš systém. Postupujeme chronologicky od raných gramatických metód cez biologicky motivované modely až po najnovšie volumetrické prístupy.

2.1 Metódy založené na pravidlách a gramatikách

Prvé systematické pokusy o modelovanie rastlín v počítačovej grafike siahajú do sedemdesiatych a osemdesiatych rokov minulého storočia. Honda [5] navrhol jednoduchý parametrický model opisujúci tvar stromu pomocou uhlov vetvenia a pomerov dĺžok po sebe nasledujúcich vetiev. Na tento základ nadviazali Aono a Kunii [1], ktorí do prístupu zakomponovali gramatický formalizmus, vďaka čomu bolo možné automaticky vytvárať pestrú škálu stromov. Oppenheimer [14] pristupoval k problému prostredníctvom fraktálnej geometrie, využívajúc sebakodobnosť vetviacich sa štruktúr.

Zásadný prelom v tejto oblasti priniesli takzvané L-systémy, ktoré pôvodne definoval biológ Aristid Lindenmayer ako formálny nástroj na simuláciu vývoja mnohobunkových organizmov. Neskôr Prusinkiewicz v spolupráci s Lindenmayerom [17] v knihe *The Algorithmic Beauty of Plants* tento koncept adaptovali a systematizovali pre oblasť počítačovej grafiky, pričom zaviedli jeho parametrické, stochastické a kontextové varianty. L-systém pozostáva z abecedy symbolov, počiatočného reťazca (axiómu) a sady prepisovacích pravidiel, ktoré sa paralelne aplikujú v každej iterácii. Výsledný reťazec sa následovne interpretuje pomocou 3D korytnačej grafiky (*turtle graphics*),

ktorá symboly prekladá na pohyby, rotácie a vetvenia.

Napriek elegancii formalizmu má klasický L-systém významné obmedzenie: pravidlá sa musia navrhnúť manuálne, čo pri zložitých druhoch stromov vyžaduje rozsiahle odborné znalosti a zdĺhavé experimentovanie. Ďurikovič [22] navrhol spôsob, ako tento problém je možné redukovať využitím pozičných dát — z nameraných pozícií kľúčových vetiev sa odvodí splajnové funkcie, na základe ktorých sa automaticky generujú L-systémové pravidlá rekonštruujúce požadovaný tvar stromu. Ako riešenie tohto problému predstavili Guo et al. [3] metódu inverzného procedurálneho modelovania, ktorá pravidlá L-systému odvodzuje zo poskytnutého 3D modelu stromu. Najnovšie trendy v tejto oblasti reprezentuje práca autorov Lee et al. [7], ktorí na automatické odvodzovanie latentných L-systémov z dát využili neurónové siete na báze transformerov.

Z uvedených prístupov preberáme najmä formalizmus L-systémov, ktorý ďalej prispôbujeme potrebám nášho modelu.

2.2 Procedurálne modelovanie

Procedurálne metódy obohacujú striktné gramatické modely o geometrické a interaktívne prvky. V tejto oblasti predstavili Weber a Penn [20] kľúčový model, ktorý definuje anatómiu stromu hierarchicky na základe úrovni vetvenia. Využíva pritom súbor parametrov, pomocou ktorých možno modifikovať tvar kmeňa, zakrivenie jednotlivých vetiev či priestorovú distribúciu lístia. Ich model sa vďaka intuitívnym parametrom stal základom mnohých komerčných nástrojov.

Alternatívny prístup predstavili Lintermann a Deussen [10], v ktorom sa strom opisuje ako graf komponentov spojených vzťahmi rodič-potomok. Výhodou tohto riešenia je efektívne prepojenie algoritmickej procedurálnej tvorby s interaktívnou modifikáciou. Ďalšie práce sa zameriavali na riešenie špecifických vlastností. Ako príklad možno uviesť prácu autorov Pirk et al. [16], ktorí simulovali interakciu stromov s okolitým priestorom, zatiaľ čo Stava et al. [19] sa zaoberali inverzným modelovaním – teda odvodzovania procedurálnych parametrov z hotových 3D modelov.

2.3 Biologicky motivované modely

Významnou kategóriou výskumu je modelovanie stromov založené na skutočných biologických procesoch. Kľúčovou myšlienkou je Shinozakiho pipe model [18] (model potrubia), ktorý hovorí, že prierez kmeňa alebo vetvy je priamo úmerný ploche listov, ktoré sú danou vetvou zásobované vodou. Z toho vyplýva jednoduché pravidlo pre výpočet hrúbky vetiev: prierez rodičovskej vetvy sa rovná súčtu prierezov jej potomkov. Tento prístup zachováva da Vinciho pravidlo a je dnes štandardnou súčasťou väčšiny

moderných modelov.

Ďalším kľúčovým biologickým konceptom je **apikálna dominancia**, teda tendencia hlavného výhonku potláčať rast bočných vetví. Borchert a Honda [2] navrhli model rozmiestnenia takzvaného vigoru (rastovej sily). V ich systéme sa táto sila distribuuje od koreňového systému do štruktúry stromu prostredníctvom parametra λ , ktorý reguluje pomer rozdelenia vigoru medzi hlavnou vetvou a jej potomkami. Tento model sa stal základom mnohých neskorších prác.

Palubicki et al. [15] predstavili inovatívny rámec samoorganizujúcich sa modelov stromov. V ich poňatí je vývoj stromu vzniká ako dôsledok lokálnych interakcií medzi pukmi a prostredím (prístupom k svetlu, konkurenciou o priestor). Tento prístup predstavuje mimoriadne elegantný kompromis medzi biologickou vernosťou a výpočtovou nenáročnosťou. Li et al. [8] tento smer ďalej rozšírili o koordinovaný model nadzemnej a podzemnej časti rastliny, v ktorom sa signály (vigor, hormóny) šíria obojsmerne cez celý rastový graf.

Náš systém preberá z tejto oblasti niekoľko princípov. Hrúbku vetiev počítame podľa pipe modelu, rast po interakcii s používateľom riadime zjednodušenou verziou Borchert-Hondovho vigor modelu s parametrom apikálnej dominancie, a zavádzame gravitropizmus a fototropizmus ako mäkké ohýbajúce sily pôsobiace na smer rastu.

2.4 Strand-based a volumetrické modely

Väčšina doteraz spomínaných metód pristupuje k stromu ako k skeletálnemu grafu, ktorého vetvy sa pri renderovaní aproximujú zovšeobecnenými valcami. Tento prístup je však limitujúci — neumožňuje to modelovať dutiny v kmeňoch, zrastené vetvy (inoskulácia), špirálovité stáčanie ani zložité bifurkácie pozorované na starších stromoch.

Prvý model založený na prameňoch (strand-based prístup) navpredstavil vo svojej práci Holton [4], ktorý strom opísal ako zväzok prameňov tiahnucich sa od listov ku koreňu. Hoci v čase svojho vzniku tento koncept ne získal širšie uplatnenie, o niekoľko rokov neskôr naň nadviazali Hädrich et al. [6], ktorí pramene použili na simuláciu lomu dreva.

Najvýznamnejšou prácou v tejto oblasti, a zároveň priamou inšpiráciou pre našu bakalársku prácu, je *Interactive Invigoration: Volumetric Modeling of Trees with Strands* od Li et al. [9]. V ich koncepte je prameň definovaný ako valec pevnej hrúbky, ktorý sa tiahne od koncového uzla skeletálneho grafu až ku koreňu. Pozície prameňov v priečných rezoch vetiev sa určujú pomocou 2D dynamiky založenej na pozíciách (position-based dynamics). Tento prístup efektívne rieši vzájomné kolízie prameňov a udržiava ich v hraniciach profilov definovaných používateľom. Výsledkom je objemová reprezentácia, ktorá umožňuje modelovať zložité tvary nedostupné pre tradičné metódy — dutiny,

špirály, zrastené kmene a staré, rozpukané vetvy.

Autori Li et al. vo svojej práci zároveň predstavili koncept interaktívnej invigorácie. Tento mechanizmus umožňuje používateľovi kliknutím na vetvu zvýšiť rastovú silu do lokálnej oblasti stromu, čím priamo ovplyvňuje ďalší rast. Systém podporuje tri hlavné operátory — invigoráciu, tvorbu adventívnych pukov (reaktívny rast po poškodení) a krútenie vetiev (*twisting*). Plná implementácia tohto prístupu však vyžaduje netriviálnu infraštruktúru pre PBD simuláciu, generovanie meshu z bodových oblakov prameňov pomocou Delaunayho triangulácie a správu používateľom definovaných profilov. Preto v našej práci preberáme koncepčnú stránku — vigor-based interaktívny rast a operátory — ale samotnú strand-based reprezentáciu ponechávame ako smer pre budúce rozšírenie.

2.5 Rekonštrukcia stromov zo vstupných dát

Alternatívnym prístupom k procedurálnemu modelovaniu je rekonštrukcia stromov zo skutočných dát — fotografií, videí či LiDAR skenov. V tejto oblasti predstavili Neubert et al. [13] metódu, ktorá odvodzuje 3D model z jednej alebo viacerých fotografií prostredníctvom simulácie toku častíc vo vnútri silutety. Livny et al. [12] navrhli prístup zameraný na rekonštrukciu stromových štruktúr priamo z nasnímaných bodových oblakov. Aktuálny výskum sa však posúva smerom k metódam hlbokého učenia. Prístupy ako DeepTree [21] a TreePartNet [11] dnes využívajú neurónové siete na komplexnú sémantickú segmentáciu a priamu syntézu stromových modelov z dostupných datasetov.

Hoci tieto metódy produkujú vysoko realistické výsledky, vyžadujú kvalitné vstupné dáta a ich výstupom je konkrétny strom zodpovedajúci vstupu, nie parametrizovaný generátor. Pre herné a simulačné aplikácie, kde potrebujeme nekonečnú variabilitu stromov riadenú intuitívnymi parametrami, zostávajú procedurálne metódy praktickejšou voľbou. Preto sa v našej práci sústredíme výhradne na procedurálny prístup.

2.6 Zhrnutie a pozícia našej práce

Kapitola 3

Teoretický základ

Táto kapitola vysvetľuje teoretické koncepty, na ktorých je postavený náš systém procedurálneho generovania stromov. Najprv definujeme základy Lindenmayerových systémov. Následne opisujeme ich interpretáciu pomocou 3D korytnačej grafiky. Ďalej zavádzame potrubný model pre výpočet hrúbok vetiev a Laplaciánovo vyhladzovanie na úpravu prechodov medzi nimi. V závere kapitoly opisujeme vigor model pre simuláciu biologicky motivovaného rastu a tropizmy — vonkajšie sily ovplyvňujúce smer rastu vetiev.

3.1 L-systémy

L-systém (Lindenmayerov systém) je paralelný prepisovací systém a typ formálnej gramatiky. Pozostáva z množiny symbolov, počiatočného reťazca a prepisovacích pravidiel, ktoré určujú transformáciu jednotlivých symbolov v každej iterácii systému.

Paralelné prepisovanie symbolov umožňuje v každom derivačnom kroku súčasne nahradzovať všetky symboly aktuálneho reťazca podľa definovaných pravidiel. Tento mechanizmus predstavuje hlavný rozdiel oproti klasickým formálnym gramatikám, ktoré fungujú na princípe sekvenčného prepisovania, pri ktorom sa počas jedného derivačného kroku prepisuje iba jeden vybraný symbol.

Paralelný charakter prepisovania umožňuje prirodzene modelovať procesy biologického rastu, pri ktorých sa bunky organizmu vyvíjajú a delia súčasne.

3.1.1 Formálna definícia

Najjednoduchšou variantou je deterministický bezkontextový L-systém, označovaný ako D0L-systém. Označenie *bezkontextový* znamená, že pri procese prepisovania symbolu sa neberú do úvahy jeho susedné znaky v reťazci, ale aplikácia pravidla závisí čisto od prepisovaného symbolu.

Formálne je D0L-systém definovaný ako trojica

$$G = (V, \omega, P), \quad (3.1)$$

kde:

- V je konečná množina nahraditeľných symbolov (abeceda),
- $\omega \in V^+$ je neprázdny počiatočný reťazec (axióma),
- $P \subset V \times V^*$ je konečná množina prepisovacích pravidiel.

Pre každý symbol $a \in V$ existuje práve jedno pravidlo $a \rightarrow \chi$, kde $\chi \in V^*$ je tzv. *nástupca* (*successor*). Ak pre symbol pravidlo neexistuje, predpokladá sa identické pravidlo $a \rightarrow a$.

Derivácia L-systému prebieha iteratívne. V každom kroku n sa na aktuálny reťazec μ_n aplikujú všetky pravidlá súčasne, čím vznikne nový reťazec μ_{n+1} . Po k iteráciách získame reťazec μ_k , ktorý opisuje štruktúru rastliny v danom štádiu vývoja.

3.1.2 Príklad derivácie

Pre ilustráciu uvažujme zjednodušený 2D L-systém prevzatý z nášho systému, kde sme z pôvodného pravidla pre listnatý strom ponechali len rotácie $+$ a $-$ v rovine s uhlom $\delta = 45^\circ$:

- Abeceda: $V = \{F, +, -, [,]\}$
- Axióma: $\omega = F$
- Pravidlo: $F \rightarrow F[+F]F[-F]F$

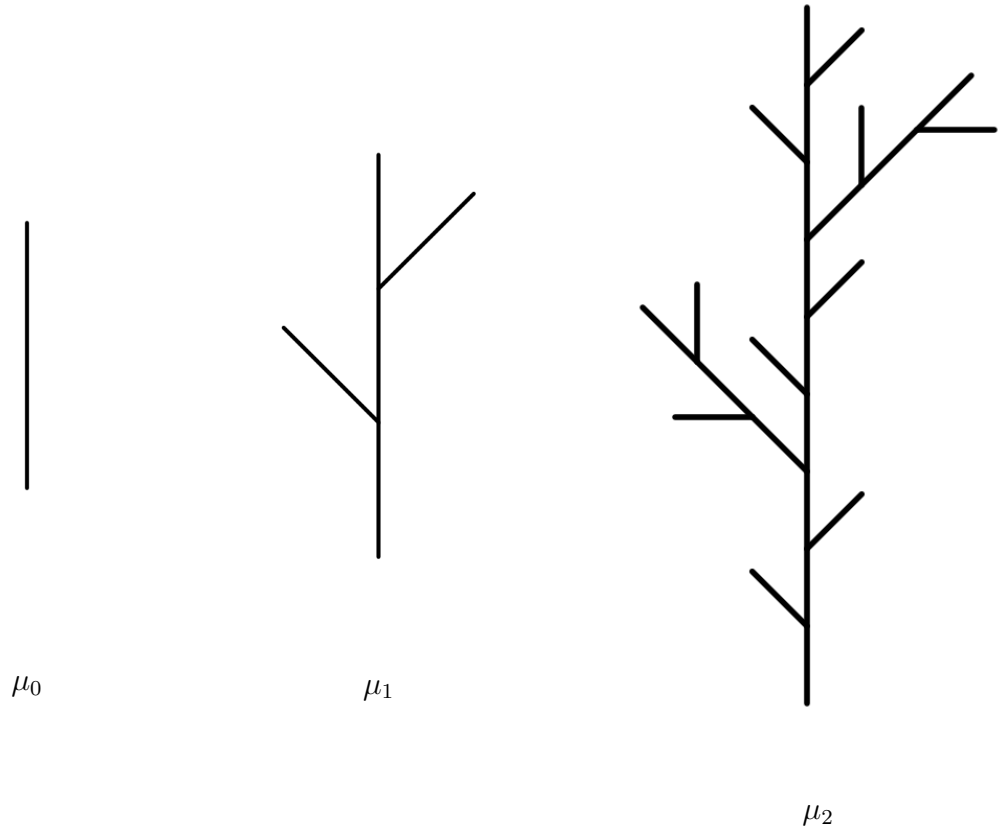
Derivácia prebieha nasledovne:

$$\mu_0 = F$$

$$\mu_1 = F[+F]F[-F]F$$

$$\mu_2 = F[+F]F[-F]F[+F[+F]F[-F]F]F[+F]F[-F]F[-F[+F]F[-F]F]F[+F]F[-F]F$$

V kroku μ_1 sa pôvodný symbol F nahradil pravou stranou pravidla, čím vznikli dve bočné vetvy a pokračujúci kmeň. V kroku μ_2 sa každý zo symbolov F v μ_1 nezávisle prepísal podľa toho istého pravidla, zatiaľ čo symboly $[,], +, -$ zostali nezmenené — nemajú definované pravidlo, preto platí identita. Vidíme, že dĺžka reťazca rastie exponenciálne s počtom iterácií. V plnom 3D systéme by pravá strana pravidla obsahovala ešte rotácie aj okolo iných osí ($\&, \wedge, /, \backslash$), ktoré určujú smer každej novej vetvy v priestore.



Obr. 3.1: Vývoj L-systému pre pravidlo $F \rightarrow F[+F]F[-F]F$ s uhlom rotácie $\delta = 45$ v 2D priestore.

3.1.3 Stochastické L-systémy

Deterministické L-systémy generujú pri rovnakých parametroch vždy identickú štruktúru. Takýto prístup je vhodný na reprezentáciu pravidelných a presne definovaných foriem, avšak prirodzené rastliny vykazujú výraznú rôznorodosť. Vetvy sa môžu rozdeliť v rôznych smeroch, mať odlišnú dĺžku alebo sa nemusia vytvoriť vôbec. Na modelovanie tejto variability sa používajú stochastické L-systémy.

V stochastickom L-systéme jednému symbolu môže byť priradených viacero pravidiel. Pri každom prepise sa jedno z nich vyberá náhodne podľa definovaných pravdepodobností.

Formálne, pre symbol $a \in V$ definujeme množinu pravidiel

$$\begin{aligned}
 a &\rightarrow \chi_1 \text{ s pravdepodobnosťou } p_1, \\
 a &\rightarrow \chi_2 \text{ s pravdepodobnosťou } p_2, \\
 &\dots \\
 a &\rightarrow \chi_m \text{ s pravdepodobnosťou } p_m,
 \end{aligned}
 \tag{3.2}$$

kde $\sum_{i=1}^m p_i = 1$. Pri derivačnom kroku sa pre každý výskyt symbolu a nezávisle vyberie jedno pravidlo podľa rozdelenia pravdepodobností. Použitím pevnej počiatočnej

hodnoty (z angl. *seed*) pseudonáhodného generátora dosiahneme reprodukovateľnosť – rovnaká počiatočná hodnota vždy vygeneruje rovnaký strom.

V našom systéme používame stochastické L-systémy s viacerými alternatívnymi pravidlami pre každý druh stromu. Napríklad pre listnatý strom existujú tri alternatívne pravidlá pre symbol F , každé s pravdepodobnosťou približne $\frac{1}{3}$, ktoré sa líšia rotáciami a poradím vetvení. Vďaka tomu vznikajú stromy s podobným charakterom, ale s individuálnymi rozdielmi v rozložení vetiev.

3.2 Korytnačia grafika v 3D

L-systémy samy osebe nevytvárajú grafickú reprezentáciu – výsledkom ich derivácie je iba postupnosť symbolov. Na prevod tohto reťazca symbolov do 3D priestoru sa používa *korytnačia interpretácia* (*turtle interpretation*), pri ktorej sa reťazec symbolov interpretuje pomocou virtuálnej korytnačky pohybujúcej sa v priestore.

3.2.1 Stav korytnačky

Korytnačka má v každom okamihu definovaný stav:

$$S = (\mathbf{p}, \mathbf{q}, l, d), \quad (3.3)$$

kde

- $\mathbf{p} \in \mathbb{R}^3$ je vektor určujúci pozíciu korytnačky v trojrozmernom priestore,
- $\mathbf{q}$ predstavuje orientáciu korytnačky reprezentovanú jednotkovým kvaterniónom,
- l je aktuálna dĺžka kroku,
- d je úroveň vetvenia.

3.2.2 Súbor príkazov

Korytnačka číta reťazec zľava doprava a vykonáva akcie podľa nasledujúcej tabuľky:

Symbol	Akcia
F	Posuň sa dopredu o l , vytvor nový uzol a segment
$+$	Rotácia okolo osi z o uhol δ (doľava)
$-$	Rotácia okolo osi z o uhol $-\delta$ (doprava)
$\&$	Rotácia okolo osi x o uhol δ (nadol)
$\wedge$	Rotácia okolo osi x o uhol $-\delta$ (nahor)
$/$	Rotácia okolo osi y o uhol $-\delta$ (roll doľava)
$\backslash$	Rotácia okolo osi y o uhol δ (roll doprava)
$[$	Ulož aktuálny stav na zásobník, zvýš d , zmenši l
$]$	Obnov stav zo zásobníka

Tabuľka 3.1: Interpretácia symbolov L-systému korytnačkou v 3D priestore.

Symbol $[$ zodpovedá začiatku novej vetvy — korytnačka si uloží svoju pozíciu a orientáciu na zásobník, zmenší dĺžku kroku o faktor λ_l (*length decay*) a zvýši hĺbku vetvenia. Po spracovaní vetvy symbol $]$ obnoví pôvodný stav. Uhol δ sa pre každú rotáciu modifikuje náhodnou odchýlkou $\epsilon \sim U(-\sigma, \sigma)$, kde σ je parameter *angle variation*, čo vnáša prirodzenú nepravidelnosť do tvaru stromu.

Výsledkom interpretácie je skeletálny graf stromu — acyklický orientovaný graf $G = (N, E)$, kde uzly N nesú informáciu o pozícii, orientácii a hĺbke vetvenia, a hrany E reprezentujú segmenty vetiev medzi susednými uzlami.

3.3 Potrubný model

Po vytvorení skeletálneho grafu je potrebné určiť polomer jednotlivých vetiev. Na tento účel sa používa potrubný model (*pipe model*), predstavený Shinozakim et al. [18]. Model vychádza z botanického pozorovania, že kmeň a vetvy stromu fungujú ako zväzok vodivých rúrok (*pipes*), ktoré transportujú vodu od koreňov k listom. Počet rúrok prechádzajúcich cez prierez vetvy je úmerný počtu (alebo ploche) listov, ktoré daná vetva zásobuje.

Matematicky to vedie k pravidlu, že prierezová plocha rodičovskej vetvy sa rovná súčtu prierezových plôch jej potomkov:

$$\pi r_{\text{parent}}^2 = \sum_{i=1}^n \pi r_{\text{child}_i}^2. \quad (3.4)$$

Keďže konštanta π vystupuje na oboch stranách rovnice, môžeme ňou vydeliť a následne odmocniť, čím získame vzťah pre polomer rodičovskej vetvy:

$$r_{\text{parent}} = \sqrt{\sum_{i=1}^n r_{\text{child}_i}^2}, \quad (3.5)$$

kde r_{parent} je polomer rodičovskej vetvy v bode vetvenia, r_{child_i} je polomer i -tej dcérskej vetvy bezprostredne nad bodom vetvenia a n je počet dcérskych vetiev vychádzajúcich z daného uzla. Mocnina 2 v rovnici 3.4 vyplýva z geometrie kruhu — plocha prierezu vetvy s polomerom r je πr^2 .

Tento vzťah je známy aj ako *da Vinciho pravidlo*, pretože Leonardo da Vinci pozoroval, že súčet hrúbok vetviev stromu zostáva približne konštantný na každej úrovni vetvenia.

V našej implementácii sa polomery počítajú rekurzívne od listov smerom ku koreňu. Koncové uzly (listy) dostanú malý počiatočný polomer r_{leaf} a polomery vnútorných uzlov sa vypočítajú podľa rovnice 3.5. Polomer koreňa je obmedzený maximálnou hodnotou r_{trunk} , ktorú nastavuje používateľ.

3.4 Vyhladzovanie polomerov

Potrubný model (rovnica 3.5) vytvára na bodoch vetvenia ostré neprirodzené skoky v hrúbke — rodičovská vetva je výrazne hrubšia než jej potomkovia. V prírode sú tieto prechody medzi vetvami plynulé, preto na výsledné polomery uplatňujeme Laplaciánovo vyhladzovanie.

Vyhladzovanie sa realizuje tak, že sa polomer každého uzla nahradí kombináciou jeho pôvodnej hodnoty a váženého priemeru polomerov susedných uzlov. Pre uzol n s aktuálnym polomerom r_n definujeme novú hodnotu r'_n ako:

$$r'_n = \underbrace{(1 - \alpha) \cdot r_n}_{\text{pôvodný polomer}} + \alpha \cdot \underbrace{\frac{\sum_{j \in \mathcal{N}(n)} w_j \cdot r_j}{\sum_{j \in \mathcal{N}(n)} w_j}}_{\text{vážený priemer susedov}}, \quad (3.6)$$

kde:

- r_n je pôvodný polomer uzla n pred vyhladzovaním,
- r'_n je nový polomer uzla n po jednej iterácii vyhladzovania,
- $\mathcal{N}(n)$ je množina susedov uzla n v skeletálnom grafe — jeho rodič a všetky jeho deti,
- r_j je polomer suseda $j \in \mathcal{N}(n)$,
- $w_j = r_j$ je váha, ktorá zabezpečuje, že hrubšie vetvy ovplyvňujú výsledok viac než tenké,
- $\alpha \in (0, 1)$ je sila vyhladzovania.

Rovnica 3.6 je lineárnou interpoláciou medzi dvomi extrémnymi hodnotami. Pri $\alpha = 0$ zostane polomer nezmenený, pri $\alpha = 1$ sa úplne nahradí váženým priemerom susedov. V praxi volíme strednú hodnotu (typicky $\alpha = 0,3$), ktorá dostatočne vyhladí prechody, ale nezmaže celkový tvar stromu daný potrubným modelom.

Vyhladzovanie sa vykonáva v niekoľkých iteráciách, pričom v každej z nich sa polomere uzlov postupne približujú k spojitkej krivke. Aby sa zachovala celková štruktúra stromu, výsledný polomer ohraničujeme zhora aj zdola voči pôvodnej hodnote $r_n^{(0)}$ z potrubného modelu:

$$r'_n \leftarrow \max \left(0,5 \cdot r_n^{(0)}, \min \left(1,2 \cdot r_n^{(0)}, r'_n \right) \right). \quad (3.7)$$

Tieto medze zabraňujú tomu, aby sa polomery v priebehu iterácií "rozutekali" – vyhladený polomer nemôže byť menší než polovica ani väčší než 1,2-násobok pôvodnej hodnoty z potrubného modelu.

Kapitola 4

Návrh systému

Kapitola 5

Implementácia

abcd implementacia

Kapitola 6

Výsledky práce

fgfggdgd

Kapitola 7

Ukážky užitočných príkazov v systéme LaTeX

V tejto kapitole si ukážeme príklady niektorých užitočných príkazov, ako napríklad správne používanie tabuliek a obrázkov, číslovanie matematických výrazov a podobne. Konkrétne príkazy použité v tejto kapitole nájdete v zdrojovom súbore `latex.tex`. Všimnite si, že pre potreby obsahu a hlavičky stránky je v zdrojovom súbore uvedený aj skrátený názov tejto kapitoly. Ďalšie užitočné príkazy nájdete aj v kapitole ??, na ktorú sme sa na tomto mieste odvolali príkazom `\ref`.

7.1 Obrázky

Vašu prácu ilustrujte vhodnými obrázkami. Pri použití programu `pdflatex` je potrebné pripraviť obrázky vo formáte pdf, jpg alebo png. Vektorové obrázky (napr. eps, svg) je najvhodnejšie skonvertovať do formátu pdf, napríklad programom Inkscape.

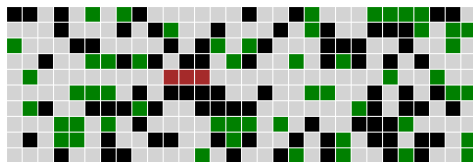
Na vkladanie obrázkov použite prostredie `figure`, ktoré obrázok umiestni na vhodné miesto, väčšinou na vrch alebo spodok stránky a tiež sa stará o automatické číslovanie obrázkov. Na každý obrázok sa treba v hlavnom texte odvolať. Napríklad ilustráciu hry Červík vidíme na obrázku 7.1. Pri odvolávaní sa na číslo obrázku používame príkaz `\ref`. Pri vložení alebo zmazaní obrázku tak nemusíme ručne všetky ostatné obrázky prečíslovať.

Podobne tabuľky vkladajte pomocou prostredia `table`, pričom samotnú tabuľku vytvoríte príkazom `tabular`. Každú tabuľku potom spomeníte aj v hlavnom texte. Napríklad v tabuľke 7.1 vidíme porovnanie časov niekoľkých fiktívnych programov.

V texte môžete tiež potrebovať dlhšie matematické výrazy, ako napríklad tento

$$\sum_{k=0}^n q^k = \frac{q^{n+1} - 1}{q - 1}. \quad (7.1)$$

Použitím prostredia `equation` bol tento výraz zarovnaný na stred na zvláštnom riadku



Obr. 7.1: Ukážka hry Červík. Červík je znázornený červenou farbou, voľné políčka sivou, jedlo zelenou a steny čiernou. Hoci tento popis obrázku je dlhší, v zdrojovom texte je aj kratšia verzia, ktorá sa zobrazí v zozname obrázkov.

Tabuľka 7.1: Doba výpočtu a operačná pamäť potrebná na spracovanie vstupu XYZ. V tomto popise môžeme vysvetliť detaily potrebné pre pochopenie údajov v tabuľke.

Meno programu	Čas (s)	Pamäť (MB)
Môj super program	25.6	120
Speedy 3.1	32.1	100
VeryOld	244.1	200

a očíslovaný. Na toto číslo sa tiež môžeme odvolať príkazom `\ref`. Napríklad rovnica (7.1) predstavuje súčet geometrickej postupnosti.

V práci tiež možno budete uvádzať úryvky kódu v niektorom programovacom jazyku. Môže vám pomôcť prostredie `lstlisting` z balíčka `listings`, v ktorom môžete nastaviť aj jazyk a kód bude krajšie sformátovaný. Ukážku nájdete ako Algoritmus 7.1.

Napokon, v texte nezabudnite citovať použitú literatúru pomocou príkazu `\cite`. Napríklad ďalšie detaily o systéme LaTeX nájdete v knihe od Tobiasa Oetikera a kolektívu [?]. Pre ukážku citujeme aj článok z vedeckého časopisu [?] a článok z konferencie [?], technickú správu [?], knihu [?] a materiál z internetu [?].

Algoritmus 7.1: Algoritmus na výpočet faktoriálu v jazyku C

```
int factorial = 1;
for(int i = 1; i <= n; i++) {
    factorial *= i;
}
```

Záver

Na záver už len odporúčania k samotnej kapitole Záver v bakalárskej práci podľa smernice [?]: „V závere je potrebné v stručnosti zhrnúť dosiahnuté výsledky vo vzťahu k stanoveným cieľom. Rozsah záveru je minimálne dve strany. Záver ako kapitola sa nečísluje.“

Všimnite si správne písanie slovenských úvodzoviek okolo predchádzajúceho citátu, ktoré sme dosiahli príkazom \uv.

V informatických prácach niekedy býva záver kratší ako dve strany, ale stále by to mal byť rozumne dlhý text, v rozsahu aspoň jednej strany. Okrem dosiahnutých cieľov sa zvyknú rozoberať aj otvorené problémy a námety na ďalšiu prácu v oblasti.

Abstrakt, úvod a záver práce obsahujú podobné informácie. Abstrakt je kratší text, ktorý má pomôcť čitateľovi sa rozhodnúť, či vôbec prácu chce čítať. Úvod má umožniť zorientovať sa v práci skôr než ju začne čítať a záver sumarizuje najdôležitejšie veci po tom, ako prácu prečítal, môže sa teda viac zamerať na detaily a využívať pojmy zavedené v práci.

Literatúra

- [1] Masaki Aono and Tosiyasu L. Kunii. Botanical tree image generation. *IEEE Computer Graphics and Applications*, 4(5):10–34, 1984.
- [2] Rolf Borchert and Hisao Honda. Control of development in the bifurcating branch system of *Tabebuia rosea*: a computer simulation. *Botanical Gazette*, 145(2):184–195, 1984.
- [3] Jianwei Guo, Haiyong Jiang, Bedrich Benes, Oliver Deussen, Xiaopeng Zhang, Dani Lischinski, and Hui Huang. Inverse procedural modeling of branching structures by inferring l-systems. *ACM Transactions on Graphics*, 39(5):1–13, 2020.
- [4] Mark Holton. Strands, gravity and botanical tree imagery. *Computer Graphics Forum*, 13(1):57–67, 1994.
- [5] Hisao Honda. Description of the form of trees by the parameters of the tree-like body: effects of the branching angle and the branch length on the shape of the tree-like body. *Journal of Theoretical Biology*, 31:331–338, 1971.
- [6] Torsten Hädrich, Jan Scheffczyk, Wojciech Palubicki, Sören Pirk, and Dominik L. Michels. Interactive wood fracture. In *Eurographics/ACM SIGGRAPH Symposium on Computer Animation — Posters*. The Eurographics Association, 2020.
- [7] Jae Joong Lee, Bosheng Li, and Bedrich Benes. Latent l-systems: Transformer-based tree generator. *ACM Transactions on Graphics*, 43(1):1–16, 2024.
- [8] Bosheng Li, Jonathan Klein, Dominik L. Michels, Bedrich Benes, Sören Pirk, and Wojtek Pałubicki. Rhizomorph: The coordinated function of shoots and roots. *ACM Transactions on Graphics*, 42(4):1–16, 2023.
- [9] Bosheng Li, Nikolas A. Schwarz, Wojtek Pałubicki, Sören Pirk, and Bedrich Benes. Interactive invigoration: Volumetric modeling of trees with strands. In *ACM SIGGRAPH 2024 Conference Papers*. ACM, 2024.
- [10] Bernd Lintermann and Oliver Deussen. Interactive modelling and animation of branching botanical structures. In *Computer Animation and Simulation '96*, pages 139–151. Springer-Verlag, 1996.

- [11] Yanchao Liu, Jianwei Guo, Bedrich Benes, Oliver Deussen, Xiaopeng Zhang, and Hui Huang. Treepartnet: Neural decomposition of point clouds for 3d tree reconstruction. *ACM Transactions on Graphics*, 40(6):1–16, 2021.
- [12] Yotam Livny, Sören Pirk, Zhanglin Cheng, Feilong Yan, Oliver Deussen, Daniel Cohen-Or, and Baoquan Chen. Texture-lobes for tree modelling. In *ACM SIGGRAPH 2011 Papers*, pages 1–10. ACM, 2011.
- [13] Boris Neubert, Thomas Franken, and Oliver Deussen. Approximate image-based tree-modeling using particle flows. *ACM Transactions on Graphics (Proc. of SIGGRAPH 2007)*, 26(3), 2007.
- [14] Peter E. Oppenheimer. Real time design and animation of fractal plants and trees. *SIGGRAPH Computer Graphics*, 20(4):55–64, 1986.
- [15] Wojciech Palubicki, Karol Horel, Steven Longay, Adam Runions, Brendan Lane, Radomír Měch, and Przemyslaw Prusinkiewicz. Self-organizing tree models for image synthesis. *ACM Transactions on Graphics*, 28(3):1–10, 2009.
- [16] Sören Pirk, Ondrej Stava, Julian Kratt, Michel Abdul Massih Said, Boris Neubert, Radomír Měch, Bedrich Benes, and Oliver Deussen. Plastic trees: Interactive self-adapting botanical tree models. *ACM Transactions on Graphics*, 31(4):1–10, 2012.
- [17] Przemyslaw Prusinkiewicz and Aristid Lindenmayer. *The Algorithmic Beauty of Plants*. Springer-Verlag, 1990.
- [18] Kichiro Shinozaki, Kyoji Yoda, Kazuo Hozumi, and Tatu Kira. A quantitative analysis of plant form — the pipe model theory: I. basic analyses. *Japanese Journal of Ecology*, 14(3):97–105, 1964.
- [19] Ondrej Stava, Sören Pirk, Julian Kratt, Baoquan Chen, Radomír Měch, Oliver Deussen, and Bedrich Benes. Inverse procedural modelling of trees. *Computer Graphics Forum*, 2014.
- [20] Jason Weber and Joseph Penn. Creation and rendering of realistic trees. In *Proceedings of SIGGRAPH '95*, pages 119–128. ACM, 1995.
- [21] Xin Zhou, Bosheng Li, Bedrich Benes, Songlin Fei, and Sören Pirk. Deeptree: Modeling trees with situated latents. *IEEE Transactions on Visualization and Computer Graphics*, pages 1–14, 2023.
- [22] Roman Ďurikovič. Towards visual modelling of bonsai trees. In *Proceedings of the Spring Conference on Computer Graphics*, 2003.

Príloha A: obsah elektronickej prílohy

V elektronickej prílohe priloženej k práci sa nachádza zdrojový kód programu a súbory s výsledkami experimentov. Zdrojový kód je zverejnený aj na stránke <http://mojadresa.com/>.

Ak uznáte za vhodné, môžete tu aj podrobnejšie rozpísať obsah tejto prílohy, prípadne poskytnúť návod na inštaláciu programu. Alternatívou je tieto informácie zahrnúť do samotnej prílohy, alebo ich uviesť na oboch miestach.

Príloha B: Používateľská príručka

V tejto prílohe uvádzame používateľskú príručku k nášmu softvéru. Tu by ďalej pokračoval text príručky. V práci nie je potrebné uvádzať používateľskú príručku, pokiaľ je používanie softvéru intuitívne alebo ak výsledkom práce nie je ucelený softvér určený pre používateľov.

V prílohách môžete uviesť aj ďalšie materiály, ktoré by mohli pôsobiť rušivo v hlavnom texte, ako napríklad rozsiahle tabuľky a podobne. Materiály, ktoré sú príliš dlhé na ich tlač, odovzdajte len v electronickej prílohe.